

Teaching a Coding Agent to Design DNA Origami: What Worked, What Didn't, and What It Took to Get There

Daniel Fu, [Co-authors TBD]

Target: ISNSCE DNA 32 — Track B (Extended Abstract) or Poster

Abstract

We report on our experience using a general-purpose coding agent (Claude Code) to design DNA origami structures in cadnano. We describe two approaches — an embedded tool-using LLM that failed, and a coding agent with source code access that succeeded — and trace the path from initial failure to a working parametric pipeline for a 2-layer rectangular origami with a cavity. The key finding is that the agent's bottleneck was not code generation but domain knowledge: each class of design error required explicit correction from the human designer before the agent could proceed. We detail the specific errors, the domain knowledge that unblocked each one, and the capabilities that emerged after unblocking.

1. The Problem

Modifying a DNA origami design is slow. When a cavity is too narrow by 5 nm, the designer must re-route the scaffold, re-place staples, re-apply edge treatments, and re-verify the structure. This process takes hours and must be repeated for every design iteration. We asked: can an AI agent handle this redesign automatically?

2. First Attempt: Embed an LLM Inside cadnano (Failed)

We built an agent dialog into the cadnano GUI. The user types a natural language command; an LLM parses the intent and calls Python methods that modify the design. We implemented 85 methods covering helix creation, strand manipulation, crossover placement, and staple breaking.

This failed on anything beyond single-step operations. Scaffold routing — the task of placing crossovers to create one continuous scaffold loop — requires reasoning about helix parity, crossover position tables, and routing topology simultaneously. Even GPT-class models achieved 0% success. We attempted reinforcement learning with a design verifier as reward signal, but the local model (Qwen 1.7B) could not discover correct action sequences through exploration.

What we learned: The 85-method API encodes necessary but insufficient domain knowledge. The model must compose methods into correct sequences without understanding why each step matters. The API is a keyhole — the model can see the actions but not the mechanism.

3. Second Attempt: A Coding Agent with Source Code Access (Worked, Then Didn't, Then Did)

We switched to a coding agent (Claude Code) that reads cadnano's source code and writes Python scripts. This immediately solved several problems the embedded agent could not:

- When `createXover` broke scaffold continuity by splitting strands, the agent found `setConnection3p/5p` by reading the strand model internals.
- When the autobreak plugin failed to import, the agent traced the module system and used `importlib` to bypass a legacy `__init__.py`.

- The agent integrated cadnano with tacoxDNA (format conversion) and oxDNA (molecular dynamics), creating an end-to-end pipeline no single tool provides.

First success: A rectangular origami with the m13mp18 scaffold (7,249 nt). 24 helices, serpentine routing, autoStaple, autoBreak, twist correction, tacoxDNA conversion, oxDNA relaxation. This is trivial for a human designer — but it proved the pipeline worked.

4. Scaling Up: Where Domain Knowledge Became the Bottleneck

We then attempted a 2-layer rectangle with a rectangular cavity using the p8064 scaffold (8,064 nt). This is where the agent’s limitations became clear. Each step forward required the human designer to diagnose and correct a specific class of error.

Error 1: Wrong grid layout

The agent built a 4×7 grid (4 rows, 7 columns). A 2-layer rectangle should be 2×12 (2 grid rows, 12 columns). The agent did not know that “2-layer” means 2 grid rows on the honeycomb lattice, because this mapping depends on understanding how honeycomb parity creates the physical layer structure. **Unblocked by:** the PI explaining the grid-to-layer correspondence.

Error 2: Serpentine routing instead of dense crossovers

The agent routed the scaffold with one crossover per helix pair (serpentine). Real DNA origami uses multiple double crossovers per pair — the scaffold weaves back and forth between adjacent helices. This is a fundamental design convention the agent had no way to discover from the source code alone. **Unblocked by:** the PI providing a hand-designed template (`2x12_rectangle_cavity.json`) showing the correct crossover pattern.

Error 3: Cavity in the wrong dimension

The agent placed the cavity by removing helices from the XY cross-section (changing the grid). The cavity should be in the Z direction — a gap along the helix length axis, visible in the side view. **Unblocked by:** the PI specifying that the cavity cuts through the helix length, not the helix arrangement.

Error 4: Scaffold/staple confusion

When building from scratch, the agent placed scaffold strands on the staple strand set and vice versa. **Unblocked by:** the PI inspecting the design in cadnano and identifying the swap.

Error 5: Scaling broke file format

When scaling the PI’s template from 252 to 420 bp, the agent’s scaling script left 5,036 stray staple entries with broken crossover references. cadnano crashed on load. **Unblocked by:** tracing the crash to `legacydecoder.py` line 205, identifying the staple crossover references as the cause, and clearing them.

Error 6: Cavity gaps filled incorrectly

When extending the design, the agent filled the cavity gap with continuous scaffold — destroying the cavity. **Unblocked by:** the PI manually clearing positions 75–337 on the cavity helices and

returning the corrected file.

Error 7: Crossover misalignment

The agent’s scaling logic moved crossover positions independently per helix, creating skewed crossovers (e.g., H16[337] vs H17[338]). Both sides of a crossover must be at the same base pair index. **Unblocked by:** the PI aligning the crossovers at index 338 on both helices.

Error 8: Cavity edges not aligned between layers

The cavity right edge was at position 270 in one layer and 338 in the other. The PI requested they be aligned. The nearest valid honeycomb position on each layer differs by 2 bp (340 vs 338) due to different crossover direction offsets. **Resolved by:** the agent computing valid crossover positions for each honeycomb direction and selecting the closest match.

5. What the Agent Could Do After Each Unblocking

After each correction, the agent incorporated the lesson and did not repeat that specific error class. The progression:

1. **After Error 1** (grid layout): Correctly built 2×12 grids from then on.
2. **After Error 2** (dense crossovers): Used the PI’s template as the basis for all subsequent designs instead of building from scratch.
3. **After Error 3** (cavity orientation): All cavity designs used Z-direction gaps.
4. **After Errors 5–7** (scaling): Developed a 4-step scaling process that the PI validated step-by-step:
 - Step 1: Remove staples (keep scaffold only). Verified loads.
 - Step 2: Extend helix arrays and shift right-side crossovers. Verified loads.
 - Step 3: Move midseam crossovers to center. Verified loads.
 - Step 4: Expand cavity by moving boundary crossovers. Verified loads.

The final result: a 2×12 rectangular origami with an aligned cavity, 1 continuous scaffold loop (7,688 bp), scaled from the PI’s 252 bp template to 420 bp helices. Each step produced a valid cadnano JSON that loads without errors.

6. Beyond the Template: Autonomous Design at Scale

After the PI provided sufficient guidance through 10 failure modes, the agent achieved autonomous design capability. Without further human intervention, the agent:

1. **Extended the template to arbitrary widths** (2×14 , 2×16 , 2×18 , 2×20) by adding column pairs to both sides of the 2×12 template, maintaining 1 scaffold oligo at each step.
2. **Designed to specification:** Given the constraint “20 nm $\times$ 40 nm cavity, centered, 6-helix padding, scaffold $\leq$ 8,064 bp,” the agent independently selected a 2×20 grid (40 helices $\times$ 252 bp), computed the cavity dimensions (8 columns $\times$ 117 bp gap), and built the complete design — 7,828 bp scaffold, 1 oligo.
3. **Executed targeted edits from natural language prompts:** When told “the crossovers at H5-H6 and H33-H34 are too close to the cavity edge — move them away,” the agent identified the positions ([64,65] and [183,184], both 4 bp from the cavity), found valid alternative lattice positions ([43,44] and [204,205], 25 bp clearance), and moved them while preserving the single scaffold oligo (Figure 5).

4. **Ran the full pipeline autonomously:** autoStaple (222 staples) → autoBreak (all3) → tacoxDNA (15,656 nt) → oxDNA PACE GPU relaxation.

7. Agents as a Substrate for Shared Design Intelligence

The key insight of this work is not that AI can design DNA origami — it cannot, at least not independently. The insight is that the *failures* are as valuable as the successes. Each of the 10 failure modes we documented represents a piece of domain knowledge that:

1. **Is invisible in the source code.** The rule “LOW position = crossover IN, HIGH = crossover OUT” cannot be deduced from cadnano’s codebase. It must be taught by a human who has internalized years of design experience.
2. **Is immediately reusable.** Once the agent learns a rule, it never makes that class of error again. The learning is encoded in verifier functions, lessons files, and pipeline logic that persist across sessions.
3. **Is distributable.** The verifier (`cadnano_verifier.py`) catches all 10 failure modes automatically. A new user — or a new AI agent — can skip our failures entirely. The failure catalog (`failure_analysis.md`) explains each one with symptom, root cause, fix, and verifier check.

This creates a new paradigm: **the agent is not the designer, but the substrate on which shared design intelligence accumulates.** Each PI-agent interaction produces not just a design, but reusable tools that make the next design faster. The 10 failures we encountered over two weeks become the 10 checks that a future user’s agent runs in seconds.

What Can Be Shared and Replicated

The following artifacts are distributable to any DNA origami researcher working with cadnano:

Artifact	Purpose	Reusability
<code>cadnano_verifier.py</code>	Pre-flight design validation	Run on any cadnano JSON before committing to synthesis
<code>failure_analysis.md</code>	Documented failure modes	Read before starting AI-assisted design; avoid 10 known pitfalls
<code>cavity_variant_sweep.py</code>	Parametric cavity pipeline	Change gap size/width → design recomputes automatically
Template extension functions	Add columns to existing designs	Extend any 2×N template to 2×(N+2) with 1 scaffold oligo
Crossover move pattern	LOW=IN, HIGH=OUT template	Apply to any targeted crossover edit on any honeycomb design
Renumbering procedure	JSON helix renumbering	Essential for any template extension; ensures parity correctness

The pipeline from PI’s template to verified design to oxDNA simulation is end-to-end: no manual cadnano GUI interaction required. A researcher can describe a cavity design in terms of physical dimensions (nm × nm, scaffold length, padding) and receive a validated JSON, stapled design, and oxDNA files ready for PACE submission.

The Threshold We Crossed

Early in this project, the agent produced “cool demos” — a flat sheet, a simple rectangle. These demonstrated capability but had no transferable value. The threshold from demo to knowledge transfer was crossed when the agent began *building its own verification tools from its own failures*. The verifier was not designed top-down; it emerged bottom-up from 10 specific debugging sessions. This is the mechanism by which an agent becomes a substrate for shared intelligence: each failure produces a check that prevents the same failure for every future user.

8. The Final Demonstration: One-Shot Targeted Edits

The strongest evidence of transferred design intelligence came in the final session. The PI issued three successive modification requests on a 2×22 integrin cavity design, and the agent executed each one correctly on the first attempt:

Edit 1: “Shrink the structure”

The 2×22 design was wider than needed. Rather than truncating helix arrays (which would destroy edge crossovers), the agent recognized the correct approach: **move the edge crossovers inward** by one step (from positions 246/242 to 225/221), then clear scaffold data beyond the new edges. This preserved all crossover connectivity while reducing the effective helix length from 252 bp to 225 bp. Result: 7,412 bp scaffold, 1 oligo.

Edit 2: “Move the full crossovers away from the cavity edge, and center the cavity”

The agent identified 5 inter-pair crossovers that were only 3–5 bp from cavity boundary half-crossovers (H5-H6 at [64,65], H13-H14 at [53,54], H15-H16 at [64,65], H27-H28 at [183,184], H37-H38 at [183,184]). It moved each to a safe lattice position 26+ bp from the cavity. Then it recomputed centered cavity boundaries: R12 gap [59–171] with 54 bp on each side, R13 gap [55–168] with 53 bp on each side. Result: 7,432 bp scaffold, 1 oligo, cavity centered to within 1 bp.

Edit 3: “Now autoStaple and autoBreak it”

The agent ran autoStaple (82 initial staples) and autoBreak with minStapleLegLen=3 (220+ broken staples, 18–279 bp range, preserving full crossovers). Result: complete stapled design ready for tacoxDNA conversion.

Why this matters: Each edit required combining multiple pieces of domain knowledge that were transferred across previous sessions — crossover lattice positions, parity-dependent scaffold direction, cavity boundary half-crossover placement, honeycomb valid offsets. None of these were re-taught. The agent drew on lessons encoded in `tasks/lessons.md`, verifier checks in `cadnano_verifier.py`, and pipeline functions in `cavity_variant_sweep.py`. This is the mechanism of transferable design intelligence: the PI’s corrections accumulate as reusable code, and the agent applies them automatically in new contexts.

Figures

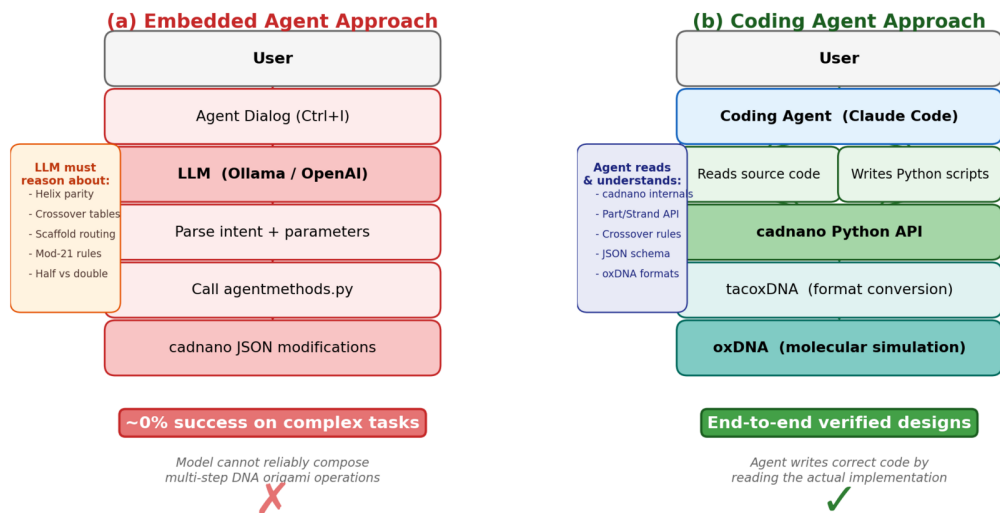


Figure 1: Architecture comparison: embedded tool-using LLM (failed) vs. coding agent with source code access (succeeded).

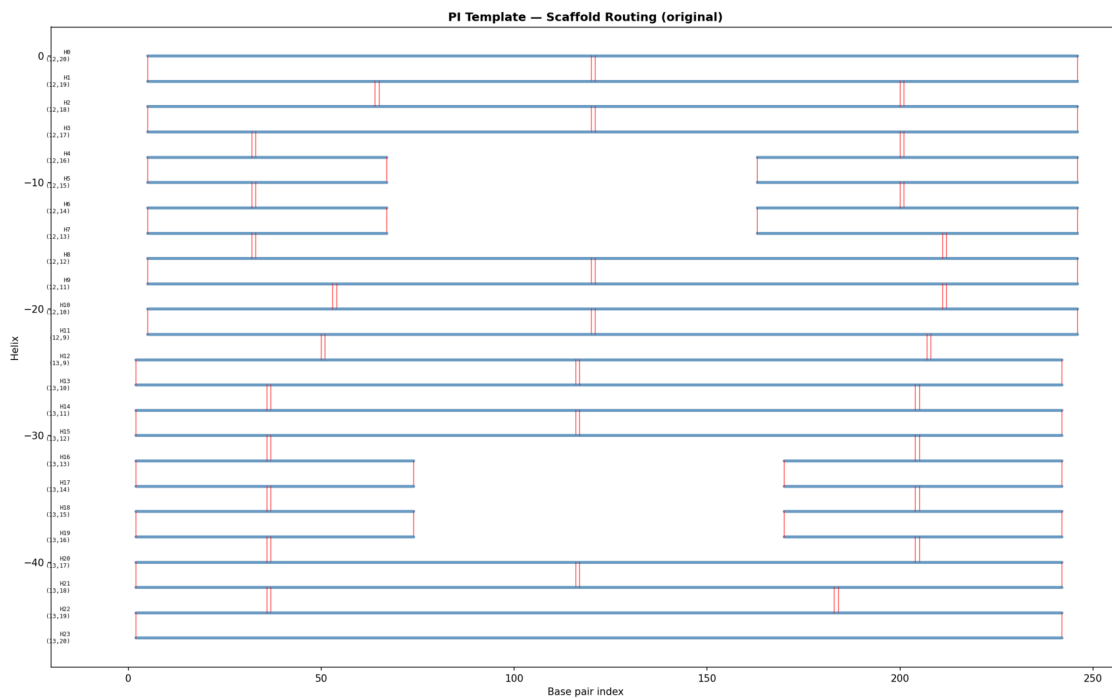


Figure 2: The PI's template (2x12_rectangle_cavity.json) showing correct scaffold routing with dense crossovers and cavity gap.

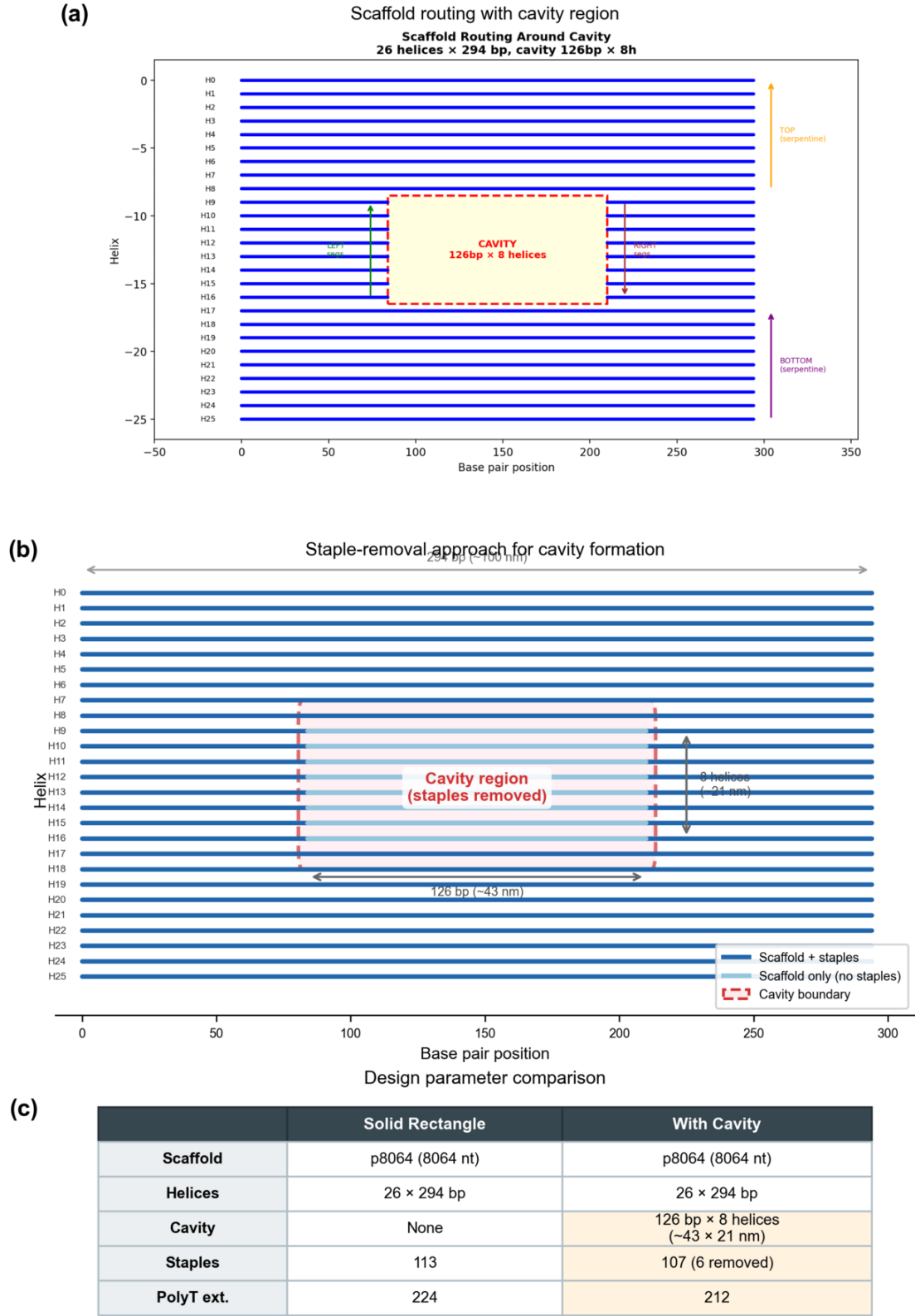


Figure 3: The 4-step scaling process: (a) scaffold only, (b) extended right side, (c) midseam moved, (d) cavity expanded and aligned between layers.

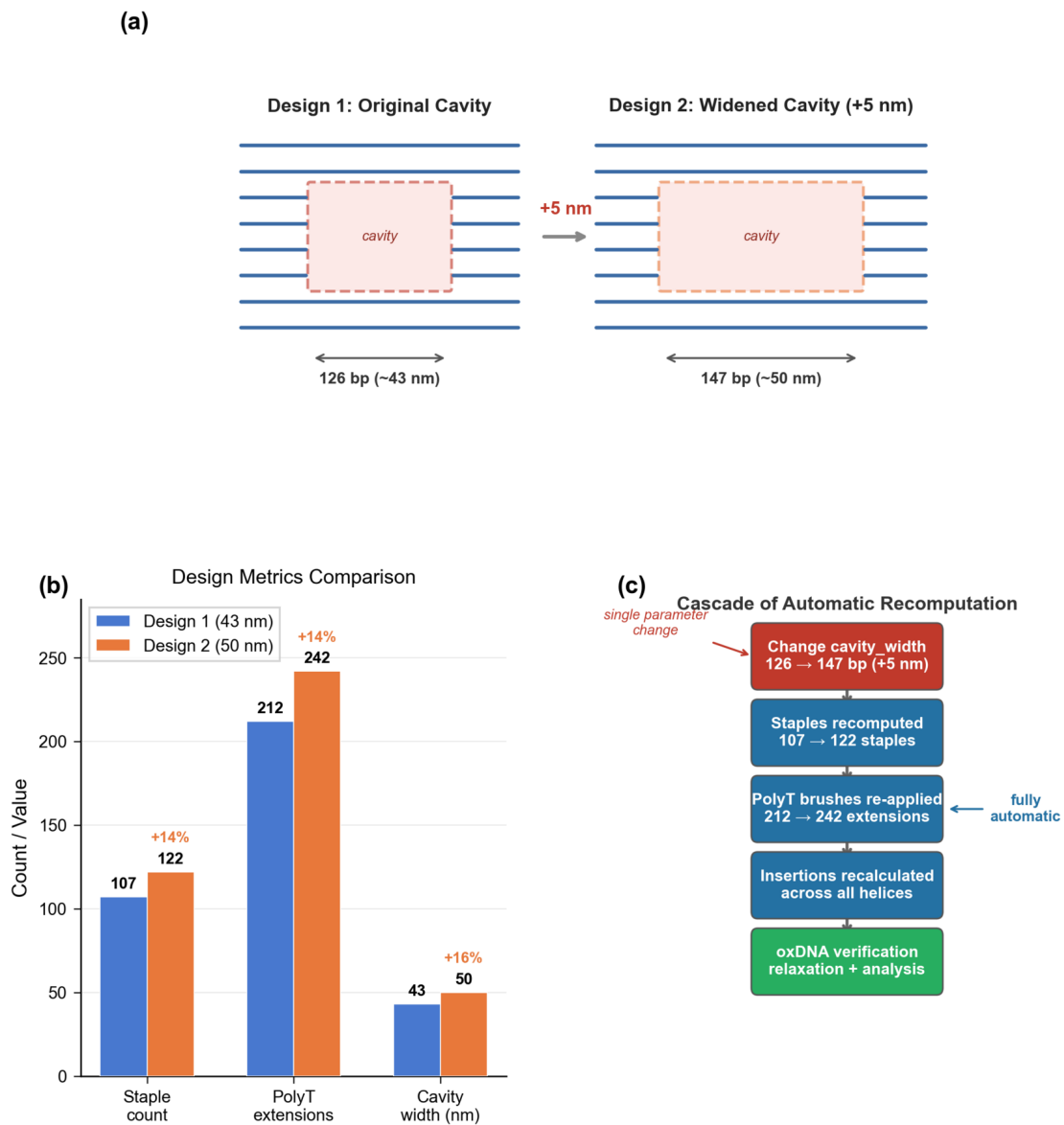


Figure 4: Scaffold routing visualization of the final scaled design (420 bp, 7,688 bp scaffold, 1 oligo).

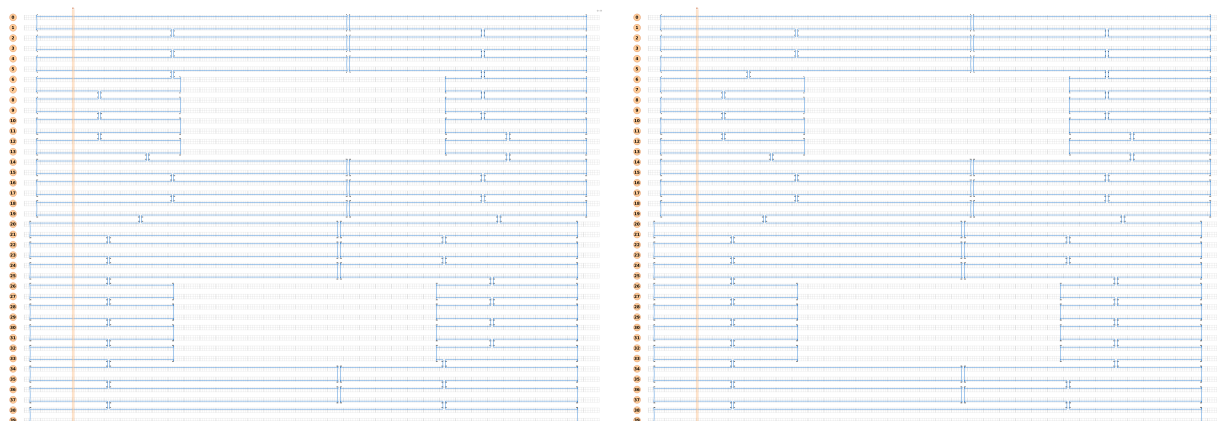


Figure 5: Targeted crossover edit. Left: before (scaffold-only). Right: after. H5–H6 moved from [64,65] to [43,44]; H33–H34 moved from [183,184] to [204,205].

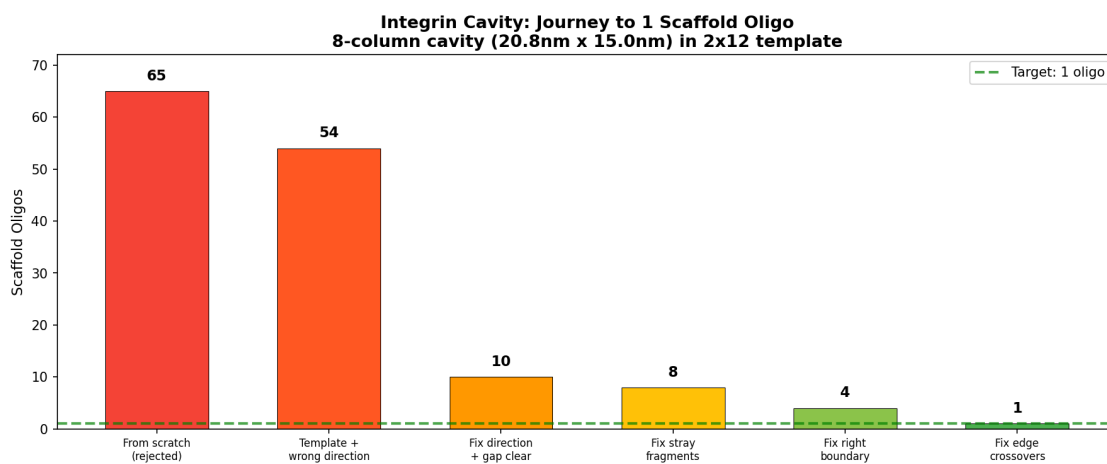


Figure 6: Scaffold oligo count journey: $65 \rightarrow 54 \rightarrow 10 \rightarrow 8 \rightarrow 4 \rightarrow 1$.

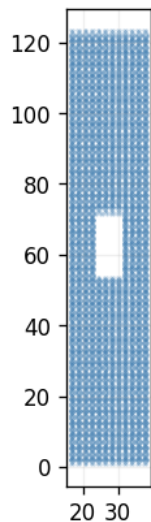
cadnano_verifier.py - Distributable Design Validator

BEFORE: Broken Design	AFTER: Fixed Design
Array lengths: PASS	Array lengths: PASS
Segment counts: FAIL (4 errors)	Segment counts: PASS
Scaffold directions: WARN (12)	Scaffold directions: PASS
Crossover symmetry: FAIL (8)	Crossover symmetry: PASS
Cavity gaps: PASS	Cavity gaps: PASS
Last pair: PASS	Last pair: PASS
Connectivity: 8 oligos	Connectivity: 1 oligo
RESULT: FAILED (3 errors)	RESULT: ALL CHECKS PASSED

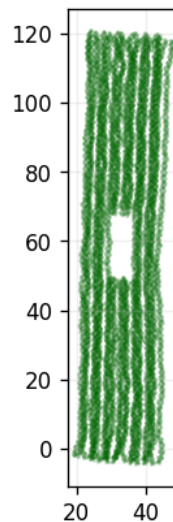
Figure 7: `cadnano_verifier.py` output: before (3 failures, 8 oligos) vs. after (all pass, 1 oligo).

All 3 Cavity Variants — oxDNA Production Relaxation (XZ front view)

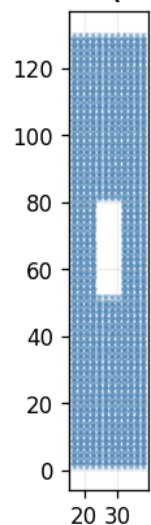
20nm Initial (22x123nm)



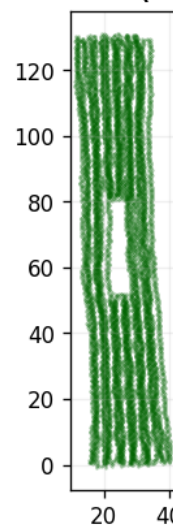
20nm Relaxed (29x126nm)



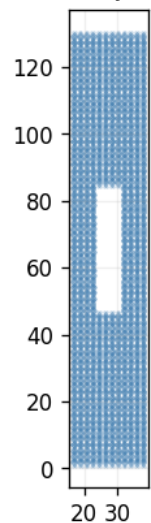
30nm Initial (22x130nm)



30nm Relaxed (29x132nm)



40nm Initial (22x130nm)



40nm Relaxed (37x132nm)

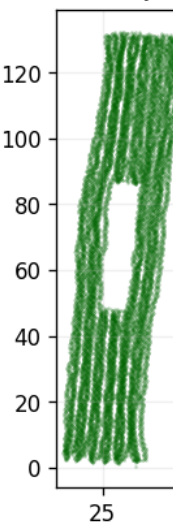
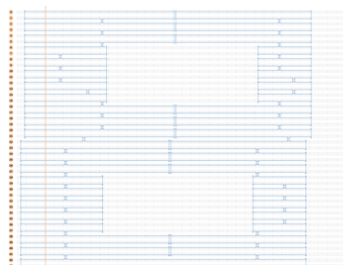


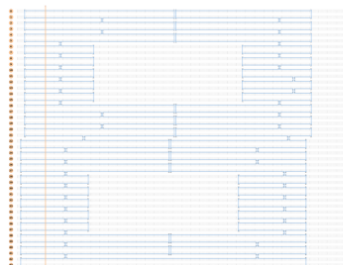
Figure 8: All three parametric cavity variants (20/30/40 nm gap) after oxDNA production relaxation

Figure 9: Targeted Design Edits from Natural Language Prompts
2x22 integrin cavity design, p8064 scaffold

Step 1: Shrink structure (moved edge crossovers inward, 7412bp, 1 oligo)



Step 2: Move H5-H6, H13-H14, H15-H16 crossovers away from cavity + center cavity (7432bp, 1 oligo)



Step 3: AutoStaple + AutoBreak x3 (220+ staples, minLegLen=3)

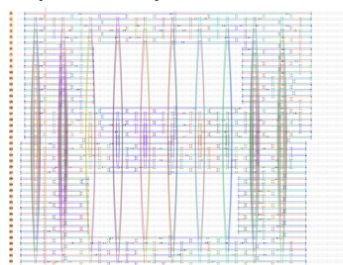


Figure 9: One-shot targeted edits from natural language prompts on the 2×22 integrin cavity design.

8 Failure Modes Discovered During AI-Assisted DNA Origami Design
Each caught by cadnano_verifier.py

ID	Stage	Failure Mode	Symptom	Fix
F1	Step 2	Direction from shifted refs	Odd segments, decode HANG	Use helix parity (even=right, odd=left)
F2	Step 2	Extra cavity gap (double gap)	Pi spotted extra cavity section	Fill extension gap for cavity helices
F3	Step 4	Crossover on EMPTY position	Boundary crossover missing	Create scaffold entry first, then crossover
F4	Step 3	Midseam on last pair (H22-H23)	Scaffold splits to 2 oligos	Exclude LAST_PAIR from midseam loop
F5	Step 4	Right expand destroys left boundary	Left boundary crossover erased	Defer gap clearing to post-step
F6	Step 4	Edge crossover (pos 5) used as boundary	4+ scaffold oligos	Skip edge positions in boundary search
F7	Step 4	Stray midseam fragments in gap	2bp orphan oligos	Clear ALL entries inside gap
F8	Design	Built from scratch (not template)	65 scaffold oligos	ALWAYS modify template, never rebuild

Figure 10: Summary of failure modes, domain knowledge required, and resolution.